CRUD Operations

Prof. Dr. Peter Braun

19. November 2024

1 Titlepage

CRUD Operations

Prof. Dr. Peter Braun

thws Technical University of Applied Sciences Würzburg-Schweinfurt

No Text – maybe some music :-)

2 Learning Goals

Learning Goals

- You learn details about four HTTP verbs
- You can map the CRUD operations to HTTP verbs
- You learn how to implement the HTTP verbs in Java

Welcome to this unit, where we will look at the next fundamental principle of Web APIs: the four basic operations for working with documents in a distributed web-based system. You have already learned the first basics of Web APIs. You know what resources are and have learned that you can address resources via URIs. You have already seen the first examples of implementing a web application in Java using the framework RESTEasy. This unit will examine the four basic HTTP verbs in detail. We will explain the possible application areas and describe the semantics, i.e., the meaning of the individual verbs and the status codes. By the way, this meaning is standardized. To some extent, you are forced to adhere to this meaning. We also explain error cases and show examples of some implementations.

3 CRUD

CRUD

- CRUD: Create, Read, Update, and Delete
- Web and REST APIs implement these operations
- The semantics of HTTP verbs is clearly defined

You already know the four basic operations needed to work with documents in a distributed system. With CREATE, you create new resources in the database; with READ, you query records from your database; with UPDATE, you update individual records; and with DELETE, you delete individual records. We abbreviate this after the first letters of the operations with CRUD. If you have already seen the last units of this course, these four operations will sound familiar. We've discussed them several times before but still need to add more details. We will map the four operations to the appropriate HTTP verbs and discuss the operations' semantics and the status codes in the response messages. Any API implemented according to Web API specifications should generally offer these four basic operations for each resource. Exceptions prove the rule here as well, of course. As developers, we are given very clear guidelines by the standards with which HTTP is defined. We must implement these four operations when implementing the server. This also gives the client's developer a clear idea of what can be done with the respective verbs. This is a significant advantage of Web-based systems: there are only these four operations, and the meaning is clear. Every developer can and may assume that the backend behaves as expected. The implementation, or rather the accurate implementation of CRUD, is just another step towards a good API. However, we will see in the upcoming units that we still have to implement many more features.

4 POST: Overview

POST: Overview

- Create a new resource on the server
- The request contains a resource representation
- The server creates the unique identifier
- Server replies with 201 Created if it was successful
- The server must reply with a `Location` header
- The server does not respond with a representation

Let's start with the POST request. We have already mentioned that a post request is used to create new resources on the server. The correct URI for creating a resource is that of the collection resource, for example, /api/persons. The client sends a representation of the resource in a format of its choice, for example, XML or JSON, in the content area of the HTTP request message. The data format in the content area must be sent to the server with the Content-Type parameter in the request message. When the client sends this post request to the server, the client does not yet know the resource's ID. The server only assigns the ID when the data is permanently stored in the database. In a few situations, the client already knows an ID before sending the HTTP request with the POST verb, for example, because the ID is a user's email address or an inventory number of a piece of furniture. We will discuss this case later when looking at the HTTP verb PUT. So now that the new record has been created in the database, the server knows the newly assigned unique ID of the new record and still has to inform the client about two things with the HTTP response. On the one hand, it must notify the client that the operation could be performed successfully. For this purpose, not the already frequently used status code 200 is used, but the status code 201, which was explicitly defined for this case with the meaning: the record was successfully created. In addition, the server must inform the client of the address to which the newly created data set can be addressed. It would be sufficient for the server to send the client the ID assigned by the database. However, if the client only received this ID, it would somehow have to know what the rest of the address, i.e., the URI, would have to be, which could link to this new ID. Therefore, the Location header should contain the full URI, including the hostname. The client can then take this URI and immediately trigger the next GET request to get the current and complete representation of the newly created resources.

5 What are the Attributes of a Resource?

What are the Attributes of a Resource?

```xml
<xs:schema
    xmlns:xs="http://www.w3.org/2001/XMLSchema"
    targetNamespace="http://www.example.org"
    elementFormDefault="qualified">
  <xs:element name="person">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="firstName" type="xs:string"/>
        <xs:element name="lastName" type="xs:string"/>
        <xs:element name="birthDate" type="xs:date"/>
        <xs:element name="email" type="xs:string"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

A common question about the POST request is related to the attributes of a resource. More precisely, how does the client know which attributes under which name and with which data type the server expects? Unlike a GET request, where the developer might even be able to figure out what attributes exist by trial and error, a POST request needs some form of description or at least an example. This could be in the form of documentation, for example. There is a standard for this called OpenAPI. Or the API provides a precise definition of the resource, for instance, in the form of an XML or JSON schema. You can see an example beside me. Such a schema then defines the names and data types of the attributes of a resource.

6 Status Codes

Status Codes

- Status 201 if the resource was created
- Status 400 if the request contained errors
- Status 401 if username/password was wrong
- Status 403 if the user is not allowed to use POST
- Status 404 if the URI was wrong
- Status 5xx, if the server cannot create the resource

Typical status codes used in a POST request: Status code 201 indicates that creating the new resorts was successful. Status code 400 indicates an error in the request that prevented the server from processing the request. A typical error case here is an error in the representation of the resource, for example, a missing comma in the case of JSON or a missing end tag in the case of XML. Status code 401 indicates that authentication is required to create a resource. The server should then return further information to the client on how successful authentication could be achieved, i.e., by which method. We will discuss this in more detail later in the HTTP security unit. If this status code is returned despite sending an authentication, the username and password combination is wrong. Status code 403, on the other hand, would indicate that the authentication was correct, but the user is still not allowed to access the resource with this POST operation. Status code 404 is relatively rare in POST because it indicates that the resource does not exist. As usual, the server responds with a status code 500 if the client's request is correct in principle, but the server still could not process the request due to an internal problem. However, status code 500 is very general and should only be used if no more specific code exists. For example, status code 503 indicates that the service is currently unavailable. The database might not be accessible, or too many client requests may have to be processed simultaneously.

7 GET: Overview

GET: Overview

- Clients request the current state of a resource
- GET does never change the state of a resource

- Collection results can be empty – this is not an error!

Next in the list of CRUD operations, we come to the READ operation. The matching HTTP verb is, of course, GET and is used by clients to request resources from the server. Such a request is directed to a whole collection of resources or a single resource if the ID is already known to the client. An HTTP request of the GET type must not change the resource on the server; after all, it is only a read access. If several GET requests are made quickly, the server must consistently deliver the same representation of the requested resource. If, in the meantime, another client has changed these resources, this restriction no longer applies, of course. Therefore, it would violate the principles of HTTP, for example, to carry a timestamp in an attribute of a resource that is also updated with each read access. This timestamp would change with every single read access, and thus, the representation would be different for two successive read accesses. But what is allowed is that you can store this attribute with the timestamp of the last read access in the database and update it with every access, as long as you don't deliver it as part of the representation with a GET request. So, for internal use, such an attribute would be allowed, but not in the representation exchanged with the client.

8 Path Template Matching

Path Template Matching

```
1  @Path("/persons")
2  public class PersonService
3  {
4          @GET
5          @Path( "{personId: \\d+}" )
6          @Produces( MediaType.APPLICATION_JSON )
7          public Response getPerson( @PathParam( "personId" ) long id ) {
8                  Person person = ... // Load resource with the given id from
                       database
9                  return Response.ok( person ).build();
10         }
11 }
```

- Regular expressions can be used in annotation `Path`
- Two methods that only differ by the regex
- Example: `"/{personId:[a-zA-Z][a-zA-Z]*}"`

Here, you can now see the source code for implementing the method for processing a GET request to a specific resource whose ID is already known. In line 5, we now use the annotation Path to request another path element. In the curly brackets, there is a placeholder under which we can access this path element's value again later in line 7. At this point, I would like to introduce you to another feature of JAX-RS that is especially relevant for GET requests. With this feature, providing additional information about allowed content in this path element is possible. For example, in line 5, you can see a regular expression that describes a sequence of digits. This regular expression is another constraint when selecting the correct method for the query type. If the last path element is a sequence of numbers, the method getPerson is called in this example. The value of this path element is then mapped to the id parameter of type long. Notice the PathParam annotation, which links the path element and the parameter. The name of the placeholder in the example personId must be identical. Otherwise, this connection will not work. In any case, if there should be a letter in this path element, the incoming request from the client will not be processed by this method. If there is no other method in our service class that could take over the processing of this request, the request will be answered with an error message. At the bottom of this slide, you see an expression describing a sequence of letters, where the first character must be a letter. So we could now look at another method with this @Path annotation and do something completely different there than in our first method, for example.

9 Status Codes

Status Codes

- Status 200, if the resource can be delivered
- Status 401, if username/password was wrong
- Status 403, if the user is not allowed to GET
- Status 404, if the resource does not exist
- Status 406, if the desired media type is invalid
- Status 5xx, in case of server errors

Status codes used 200 is the most frequently used in the case of GET. It indicates that the request was correct and could be processed completely. In the HTTP response message, the content area then represents the requested resource or resources. However, the client could also send an HTTP GET request to a URI that is not valid on the server. For example, path elements could be incorrect, or the specified identifier could be invalid. Perhaps the resource that the client wants to request here was deleted by another client shortly before. The server then responds to such error cases with the status code 404, and we recognize from the value range that the error in this request was with the client. The client has used the wrong address. And it generally makes no sense for the client to try again in the future to request a resource at this address. The client can use a parameter in the request to specify the representation format in which the server should send the desired resource. It is also possible for the client to pass a list of several representation formats here, from which the server can then select a format. We'll talk about these media types in more detail in a later unit, and then you'll learn that the client can even add weight when submitting media types. So, for example, it can express that it would prefer JSON, but it would also be fine with XML in a pinch. The server must consider these client requests, but if it cannot provide the desired representation format, it may answer the request with an error code. The correct status code for this is 406, indicating a serious problem. We have a communication problem between the client and server; the client wants the resources in a specific format, and this specification is probably hardcoded into the client, but the server is not or can no longer provide this representation format. No matter how often the client sends the same request to the server again, the server cannot answer this request successfully. As usual, the status code 500 is meant to tell the client that there was a serious problem on the server while

processing the request.

10 Exception Handling

Exception Handling

```
1  @Path("/persons")
2  public class PersonService {
3     @GET
4     @Path( "/{personId: \\d+}" )
5     @Produces( MediaType.APPLICATION_JSON )
6     public Response getPerson( @PathParam( "personId" ) long id ) {
7        Person person = ... // Load resource with the given id from database
8        if( person == null )
9              throw new WebApplicationException(Response.Status.
                     NOT_FOUND);
10       }
11 }
```

- Throw an instance of a `WebApplicationException`

So far, in the examples, I have never shown you how to use Jersey to generate status codes other than 200 or 201. However, with GET requests, it is relatively common for you to answer with a status code 404, so I would like to show you how to implement something like that here. For example, in implementing the method to query a single person, the database access would occur at line 7. Let's assume that the database does not find any record under the given ID, and then it will reply with "nullät that point. We intercept the case in line 8 and now generate a status code 404 as a response message via the exception. However, this is only one way; you can replace the status code from the Response class with a status() method and return the Response itself. However, in both cases, please ensure you do not work with 404 but with the constants provided.

11 PUT: Overview

PUT: Overview

- PUT can be used to create or modify a resource
- The URI must identify the resource uniquely
- The request contains a complete representation
- You can use verb `Patch` to partially update
- If it is not allowed to update a resource, you should not provide an implementation for PUT

The third CRUD operation is responsible for updating resources. The corresponding HTTP verb is PUT. The client sends an HTTP request message to the URI of the resource to be updated and sends a representation of the updated resource in the content area of the request message. The URI must, of course, contain the ID of the resource to be updated. This representation must be complete; that means it must include all the attributes of the resource because the server takes the updated resource from the content area of the request message and writes all the attributes to the database in order. For example, it would not be permissible for the client to send along only a representation of the actual updated attributes. There is a separate HTTP verb called PATCH for this case, which we will not consider further here, but you are allowed to use it. There are also situations where modifying resources should not be permitted. This can be the case in general; for example, updating a user's image here doesn't generally make much sense. You would rather create a new image and delete the old one. In this case, you should also not provide an implementation for the HTTP verb PUT for this resource. RESTEasy would then automatically respond to you to incoming requests with this HTTP verb accordingly with an error code informing the client that this operation is not allowed. It may also be that updating should be possible for the resource in general. Still, it may not be possible currently only due to certain constraints defined as a status in the resource, for example.

12 POST vs. PUT to Create New Resources

POST vs. PUT to Create New Resources

- Another ongoing and never-ending discussion

- `POST`, if the server defines the identifier
- `PUT`, if the client knows the identifier already

However, the HTTP verb PUT may also be used to create a new resource, which is now confusing. We initially said new resources are always created with the HTTP verb POST. But there is a small difference, which is easy to understand. A POST request may only be made if the client does not yet know the resource's identifier; this is the standard case when creating a new resource. However, if within the application, the client should see the ID of the resource even before it is made on the server, for example, because it is the e-mail address of a user or the inventory number of a piece of furniture, then the client may, and in fact, must send a PUT request to store this new resource on the server. So whenever the client already knows the resource's ID, it uses PUT to create and update it. If the client does not know the ID, it must make a POST request. It's not that hard, but in practice, this is unfortunately often done incorrectly, and also, on Stackoverflow, you will find many wrong answers about this. Please do not get confused.

13 Status Codes

Status Codes

- Status 200, respond with the new representation
- Status 201, if it was created successfully
- Status 204, if the response doesn't contain resource

- Status 400, if the request contained errors
- Status 401, if username/password was wrong
- Status 403, if the user is not allowed to PUT
- Status 404, if it does not exist
- Status 405, if it must not be updated
- Status 409, if it was updated by another client

- Status 5xx as usually

The list of status codes is now a bit longer here. There are two options in the case of PUT to indicate to the client that the request was successfully processed. You can return a status code 200, indicating to the client that the operation was successful. However, in that case, you must also submit the updated representation in the content area of the HTTP response message to the client. This can be useful in some situations, for example, if the client should continue working with the resource immediately. However, the normal case is that the request should be answered with the status code 204, which means the update was successful. Still, the server does not respond with a representation of the newly updated resource, but the content area of the response message is simply empty. If, contrary to expectations, the client wants to continue working with the updated resource immediately, it may have to request it again via a GET request. So, 200 if the server responds with the representation and 204 if the response should be empty. If the PUT operation was also used to create a resource, the status code 201 must be used as soon as the processing was successful. Code 404 indicates, as usual, that the addressed resource does not exist or no longer exists. Status code 405 informs the client that the resource is available in principle, but an update is not allowed. Status code 409 leads us to a crucial and very interesting error case. Please imagine that two clients are working practically simultaneously with the same resource. The first client now sends its update to the server. However, the second client is still working with the outdated resource, modifies it, and then sends a request to the server to update it. So, without further precautions, the second request would win, i.e., the update of the second client is in the database and has overwritten the update of the first client. We will look at possible solutions to this problem in a later unit on caching. But let's assume that the server already has an answer here; it would use status code 409 to inform the second client that

the first client has already successfully updated the resource.

14 DELETE: Overview

DELETE: Overview

- Delete a resource by sending a request
- Clients don't need the resource anymore
- Depending on the application, this could mean:
  - A message is deleted
  - A profile image is replaced by a default image
  - An order is canceled but remains in the database
- Depending on the resource type, deleting might be prohibited

The last HTTP verb or CRUD operation is now DELETE. The client sends such a request to the server to delete a specific resource. The URI here must also contain the resource ID. The content area of the HTTP request message is typically empty. From the client's point of view, such a request is transmitted to the server for deletion when the client believes it no longer needs these resources. However, it depends greatly on the use case and what this means. Let's look at a few examples. If a user wants to delete a message posted on social media, then this message should be deleted from the database. For instance, if a user on Instagram wants to delete their profile picture, this could also involve displaying a default picture for that user. If an order on Amazon is to be deleted, it could make sense that it remains in the database but is marked as deleted. Under certain circumstances, it may also make sense to prohibit the deletion of a resource. For example, you could define that an order already being processed at Amazon cannot be undone after a certain point.

15 Status Codes

Status Codes

- Status 204 is the most used reply
- Status 401, if username/password was wrong
- Status 403, if the user is not allowed to delete
- Status 404, if the resource does not exist
- Status 405, if the resource must not be deleted
- Status 5xx, as usual, in case of server problems

Typical status codes that the server sends back to the client in the response message could be the following: with status code 204, the server indicates that the delete operation was successful, but of course, it no longer transmits a representation of the deleted resources to the client. Therefore, status code 204 must be used, not status code 200. If the resource to be deleted does not exist, the server usually responds with a status code 404, but to indicate to the client that deletion is not allowed, it would choose status code 405. Status code 500 is used as usual if there are technical problems with the deletion on the server side.

16 Safety

Safety

- A verb is safe if it does not alter the resource's state

At the end of this unit, we take a closer look at the four most important HTTP verbs and their meanings. Now, let us consider two fundamental properties of these HTTP verbs. The first property is safety, but not in the sense of IT security but in the sense of changing a resource. The question is: does access cause side effects that the client must be aware of? Or are there no side effects, i.e., the access is purely reading access? This property is called safety, and an HTTP verb is safe if it does not change the resource's state. This means that there must be no side effects. This does not mean that the state of the database will not change due to this operation. It is about the representation of the resource, not the stored data. We had already discussed this when we discussed whether there might be an attribute "last access timeör not. We said the server might carry such a timestamp internally, but it must not be part of the representation. Also, the server can send out log messages; nothing concerns the client. So why is safety important? If an HTTP verb has to be safe or secure according to the specification, it is like a contract between client and server that if this HTTP verb is used, nothing in the resource will be changed. The client can request this resource repeatedly and get the same response unless another client has changed the resource. The client can also be sure that no unexpected actions will be performed on the server due to the access, which the client may not have intended. And at the same time, this contract says that the server must implement this HTTP verb similarly. Think for yourself which of the four HTTP verbs we have discussed today should have the property of being safe or secure. You should pause the video now.

17 Safe HTTP Verbs

Safe HTTP Verbs

- **GET is safe** – it's a read-only request
- POST is not safe – a resource is created
- PUT is not safe – a resource is modified
- DELETE is not safe – a resource is deleted

According to this definition, only the HTTP verb GET is safe. With GET access, the server is not allowed to change anything in the resource, and the client can rely on the fact that no page effects occur. With POST, we create new data; with PUT, we update data, and with the verb DELETE, we change data. These three operations are meant actually to modify the resource. Therefore, these three HTTP verbs are not safe.

18 Idempotence

- Idempotence means that many identical requests will have the same outcome

The second property is the so-called 'idem'potence. With this, we want to describe that no other result can be generated by reprocessing the same request on the server. So, a query is idempotent in this sense if you can execute it any number of times in succession without any difference in the result. This does not mean that the server must always respond identically to these queries – it is about the effect or the result. For example, what happens in the database. The question of why this is important. It is a contract between the client and the server. The best way to understand the importance of this is to consider an error case. Let's say the client sends a request to the server and gets no response. Can the client now send the request again without any danger? This is manageable for an idempotent request. The effect must be the same. But if the HTTP verb is not idempotent, then resending the same request may cause a problem. Please first think which of the four HTTP verbs we discussed today should have the property of being idempotent. You should pause the video now.

19 Idempotent HTTP Verbs

Idempotent HTTP Verbs

- **GET is idempotent** – it's a read-only request
- POST is not idempotent – a resource is created
- **PUT is idempotent** – a resource is modified again
- **DELETE is idempotent** – a resource is deleted

With the HTTP verb GET, a resource can be requested multiple times; we won't have a problem there. Even the numerous updates with the verb PUT of a resource are fine here. With the second request, we will update the resources again similarly. It may sound surprising that the HTTP verb DELETE is also idempotent. Perhaps the easiest way to explain it is this: after deleting a resource, nothing wrong can happen by reprocessing a delete operation; after all, the resource has already been deleted. Note that an idempotent verb does not always need to send the same response message- only the effect must be the same. A DELETE request on an already deleted resource will be answered with a status code 404, but that is irrelevant. The only HTTP operation that, by definition, does not have to be idempotent and usually cannot be idempotent is, of course, the POST operation. With the POST verb, we create new data, and it makes a difference whether a resource has been created only once or multiple times. In this context, you should always consider the bank transfer example. When you trigger a bank transfer by a POST, resending the same request triggers the transfer multiple times. This is usually different from what is wanted.

20 Summary

Summary

- You know the meaning of four HTTP verbs.
- You can map CRUD operations to HTTP verbs.
- You know implementations of the four HTTP verbs.
- You know the meaning of safety and idempotency

So, that's it again for today. Today, you got a deeper insight into the HTTP verbs GET, POST, PUT, and DELETE. You have seen that these four verbs can represent the typical CRUD operations we know from databases. For each of the four verbs, we have shown you initial implementations in Java using the Jersey framework. Finally, we discussed safety and idempotency as two crucial properties of the four HTTP verbs. Knowing how the four verbs work and their meaning, you can work successfully with an API, even if you still need to read the documentation. Using the standards, you can assume that a server will behave as expected.

21 Literature

Literature

- Bill Burke: RESTful Java with JAX-RS 2.0: Designing and Developing Distributed Web Services (Englisch). O'Reilly, 2013.